

Introduction to ROS 2

Study Notes

Contents

1	Overview	2
2	Understanding ROS as a Robotics Framework	2
2.1	What Is ROS?	2
3	Important Features of ROS	2
3.1	The ROS Equation	3
4	ROS Development Timeline	3
5	Comparison of ROS 1 and ROS 2 Architecture	4
5.1	OS Layer	4
5.2	Middleware Layer	5
5.3	Application Layer	5
5.4	Feature Comparison Table	5
6	Why Migrate from ROS 1 to ROS 2?	5
7	Diving into DDS in ROS 2	6
7.1	What Is DDS?	6
7.2	Components of the DDS Standard	6
7.2.1	Platform Layer	6
7.2.2	Middleware Layer	6
7.2.3	Application Layer	7
7.3	How DDS Works	7
7.4	Example Pub-Sub Workflow	7
8	List of DDS Vendors	7
9	Layers Built on Top of DDS in ROS 2	9
9.1	RMW Layer (ROS Middleware Abstraction Interface)	9
9.2	RCL Layer and Client Libraries	10
9.3	ROS 2 Application Layer	10
10	Summary	10

1 Overview

Robots consist of three main sections: **sensors**, **actuators**, and a **computer**. The computer acts as the brain of the robot, running the software that makes it intelligent and adaptable. The **Robot Operating System (ROS)** is a popular software framework used to program robots efficiently, widely adopted in both academia and industry for modeling, simulating, and developing robotics applications.

This document covers:

- ROS as a framework for robotics
- Important features of ROS
- Comparison of ROS 1 and ROS 2
- Communication stack in ROS 2
- DDS vendors in ROS 2

2 Understanding ROS as a Robotics Framework

Developing complex robotics applications from scratch is tedious, and if software is not modular or reusable, others must repeat the same work. This is why software frameworks and libraries exist – they provide ready-made functions or APIs of commonly used algorithms that can be integrated into new applications.

ROS is one of the most popular such frameworks, allowing developers to quickly build robotics applications by reusing code already written by others.

2.1 What Is ROS?

Despite its name, ROS is **not** a real operating system like Windows, Linux, or macOS. It has some OS-like functionalities but is fundamentally an open-source **framework** for developing robot software applications. It provides:

- Robot-specific libraries
- Software tools
- Implementations of high-level robot capabilities

ROS software is distributed in **packages**, which developers can create, share, and reuse. Each package may contain algorithm implementations, datasets, robot configuration files, robot models, etc.

There are two major versions:

- **ROS 1** – legacy version, fully supported only on Linux-based OSs.
- **ROS 2** – latest version, compatible with Windows, Linux, and macOS.

The ROS project is developed and maintained by **Open Robotics**, working with a global community of developers, researchers, and industry professionals.

3 Important Features of ROS

1. **Inter-Process Communication (IPC) Infrastructure:** Robot programs (“ROS nodes”) communicate with each other rather than being written as one large monolithic program, which reduces complexity and simplifies debugging. ROS provides APIs for this communication. ROS 2’s IPC infrastructure differs from and improves upon ROS 1’s.

2. **Reusability and Modularity:** ROS has more than 10,000 community-contributed packages, each modular and reusable, saving significant development time (e.g., the `laser_scan_merger` package for combining laser scans).
3. **Open-Source Licensing:** Most ROS software uses the **BSD 3-Clause** license, allowing modification and redistribution for commercial use. Some packages instead use Apache 2.0, MIT, or LGPL licenses – always check individual package licenses.
4. **Tools and Libraries:** ROS provides tools for visualizing sensor data and debugging inter-node communication. Client libraries such as `rclcpp` (C++) and `rclpy` (Python) enable node communication, while libraries like **TF (Transformation)** track the transforms of robot links and joints.
5. **Hardware Abstraction:** A ROS driver is a set of nodes that interface with hardware (actuators, sensors, embedded platforms). ROS’s standard interface makes it easy to swap hardware components.
6. **Ready-to-Use Robot Capabilities:** ROS provides ready-made templates for high-level tasks like autonomous navigation, manipulation, and perception, which developers can customize.
7. **Cross-Platform Support:** ROS 1 mainly supports Linux (e.g., Ubuntu); ROS 2 supports Ubuntu Linux, Windows 10, and macOS.
8. **Middleware Independence:** The middleware is the communication layer between user applications and the OS. ROS 1 used one fixed middleware, while ROS 2 uses a **pluggable** middleware infrastructure – **DDS** (Data Distribution Service) or **Zenoh** (Zero Overhead Network Protocol). Examples of DDS implementations include Fast DDS, Cyclone DDS, and RTI Connex.
9. **Community and Ecosystem:** A huge global community contributes packages, forums, and blogs. **ROSCon** is the annual community event.
10. **Testing and Simulation:** ROS includes testing frameworks and integrates with simulators such as Gazebo, Webots, and Isaac Sim.
11. **Production-Ready Framework:** ROS 1 was designed mainly for research; ROS 2 is designed for production use across industries such as automation, healthcare, and autonomous vehicles.

3.1 The ROS Equation

The important components of ROS 1 and ROS 2 can be summarized as:

- **Plumbing:** Refers to IPC facilitated by ROS communication middleware. ROS 1 uses **TCPROS** (TCP-based); ROS 2 uses **DDS**.
- **Tools:** GUI tools (e.g., **RViz**, **rqt**) and command-line tools for visualization and debugging.
- **Capabilities:** Navigation (**Nav2**), Manipulation (**MoveIt**), and Perception (integrated with OpenCV, PCL, BT.CPP, Open3D).
- **Community:** A large, active open-source community with platforms such as ROS Discourse, Robotics Stack Exchange, and ROS Discord.

4 ROS Development Timeline

Year	Milestone
2007	ROS development began at Stanford AI Laboratory; later expanded by Willow Garage.
2008	Early version 0.4 (<i>Mango Tango</i>) released by Willow Garage.
2010	First official ROS 1.0 distribution, <i>Box Turtle</i> , released; also <i>C Turtle</i> .
2011	<i>Diamondback</i> and <i>Electric Emys</i> released.
2012	<i>Fuerte</i> and <i>Groovy</i> released.
2013	<i>Hydro Medusa</i> released with major package/documentation updates.
2014	<i>Indigo</i> released; OSRF begins maintaining ROS after Hydro.
2015	<i>Jade Turtle</i> released.
2016	<i>Kinetic Kame</i> released.
2017	<i>Lunar Loggerhead</i> (ROS 1) and <i>Ardent Apalone</i> – the first ROS 2 distribution – released.
2018	ROS 1 <i>Melodic Morenia</i> and ROS 2 <i>Bouncy Bolson</i> released.
2019	ROS 2 <i>Dashing</i> and <i>Eloquent</i> released.
2020	ROS 1 <i>Noetic Ninjemys</i> (final ROS 1 release, supported until 2025) and ROS 2 <i>Foxy Fitzroy</i> released.
2021	ROS 2 <i>Galactic</i> released.
2022	ROS 2 <i>Humble Hawksbill</i> released.
2023	ROS 2 <i>Iron Irwini</i> released.
2024	ROS 2 <i>Jazzy Jalisco</i> released (EOL 2029) – current course distribution.
2025	ROS 2 <i>Kilted Kaiju</i> released.

5 Comparison of ROS 1 and ROS 2 Architecture

Both architectures consist of three main layers: the **OS Layer**, the **Middleware Layer**, and the **Application Layer**.

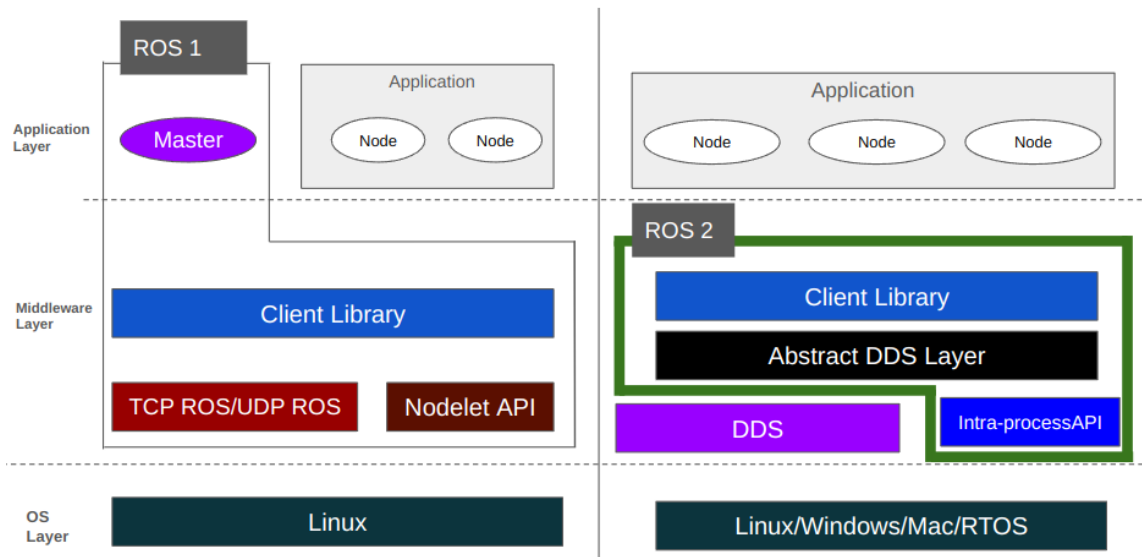


Figure 1: Architectural comparison of ROS 1 (left) and ROS 2 (right).

5.1 OS Layer

ROS 1 is mainly compatible with Ubuntu/Linux, while ROS 2 supports Linux, Windows, macOS, and RTOS.

5.2 Middleware Layer

- **ROS 1:** Custom-built middleware – **TCP ROS / UDP ROS** and the **Nodelet API** (used for intra-process communication – fast, zero-copy data transfer between threads/modules within a single process, as opposed to inter-process communication between separate processes).
- **ROS 2:** Uses **DDS** as its communication middleware, already proven in production-grade autonomous robotics, aerospace, and defense systems. An **Intra-process API** replaces the Nodelet API, and a new **Abstract DDS Layer (RMW)** sits between DDS and the client libraries, decoupling ROS 2 core libraries from the specific DDS/Zenoh implementation in use.

Client libraries provide APIs for node communication in different languages: **roscpp/rospy** (ROS 1) vs. **rclcpp/rclpy** (ROS 2). In ROS 2, all client libraries are built on top of the **ROS Client Library (RCL)**, avoiding the need to build each client library from scratch.

5.3 Application Layer

- **ROS 1:** Requires a **master node** – a broker program that must be running for nodes to discover and communicate with each other. If the master fails, the entire application is affected (a single point of failure).
- **ROS 2:** Has **no master node**. Nodes discover each other directly via DDS.

5.4 Feature Comparison Table

Feature	ROS 1	ROS 2
Communication Middleware	Custom TCP-based; master node for registration/lookup (single point of failure)	DDS and protocols like Zenoh
Real-Time	Not designed for real-time; needs significant tweaking	Real-time capable with modest tweaking
Security	No built-in security; less suited to sensitive applications	Built-in DDS security: authentication, encryption, access control
Modularity & Scalability	Highly modular, multi-language support	Enhanced modularity, multi-language, more flexibility
Platform Support	Primarily Linux	Windows, Linux, macOS
Community & Ecosystem	Vast, established; mainly academic research	Rapidly growing; community shifting from ROS 1 due to architectural limits

6 Why Migrate from ROS 1 to ROS 2?

- **ROS 1 Noetic Ninjemys** (final ROS 1 release) reaches **EOL in 2025** – no further features, bug fixes, or security updates. Its primary OS, **Ubuntu 20.04**, also reaches EOL in 2025.
- **ROS 2** suits both research and commercial applications, whereas ROS 1 was designed mainly for academic/research use.
- ROS 2 outperforms ROS 1 on security, real-time capability, Quality of Service (QoS), and scalability.

- ROS 1 is essentially Linux-only (unofficial/unstable ports exist for Windows/macOS); ROS 2 is natively cross-platform.
- For existing ROS 1 developers, the learning curve is relatively low since core communication concepts remain similar – changes are mainly at the API level.

7 Diving into DDS in ROS 2

7.1 What Is DDS?

DDS (Data Distribution Service) is a standard from the **Object Management Group (OMG)**, established in 2004 for standardized publisher/subscriber communication. It is used in mission-critical systems such as air traffic control, financial trading, and telemetry, enabling scalable, real-time, dependable, high-performance data exchange.

Use Cases:

- **Aerospace & Defense:** Command-and-control systems, real-time UAV communication.
- **Healthcare:** Patient monitoring, health information systems.
- **Transportation:** Autonomous vehicles, railway signaling/control.
- **Finance:** High-frequency trading, market data distribution.
- **Robotics:** Autonomous and industrial robots.

7.2 Components of the DDS Standard

DDS has three layers: **Platform**, **Middleware**, and **Application**.

7.2.1 Platform Layer

The foundation layer – OS, network protocols, and interfaces (e.g., Ethernet):

- **IP:** Network-layer rules for routing data.
- **Transport Protocols:**
 - **TCP** – reliable, ordered delivery.
 - **UDP** – minimal latency, used in real-time robotics/aerospace/defense.
 - **TLS** – secure TCP communication.
 - **TSN (Time Sensitive Networking)** – real-time communication.
 - **Shared Memory** – high-speed, low-latency transfer for participants on the same machine.

7.2.2 Middleware Layer

- **RTPS (Real-Time Publish-Subscribe):** Ensures interoperability across DDS implementations; designed for real-time, critical applications.
- **DDS Core Features:**
 - **Data-Centric Publish/Subscribe Model** – decouples publishers and subscribers.
 - **QoS Policies** – configure reliability, durability, latency, resource usage.
 - **Interoperability** – platform-independent, standardized APIs.
 - **Real-Time Communication** – low latency, high throughput, deterministic delivery.
 - **Scalability** – dynamic discovery of publishers/subscribers.
 - **Security** – authentication, encryption, access control.

- **Language-Specific Implementations** – DDS-C++, DDS-Java, DDS-IDL-C, DDS-IDL-C#, etc.
- **Additional DDS Specifications:**
 - **DDS-WEB** – integrating web technologies.
 - **DDS-OPC UA** – compatibility with OPC Unified Architecture.
 - **DDS-RPC** – remote procedure call support.
 - **DDS-XTYPES** – complex data modeling.
 - **IDL 4** – version of the Interface Definition Language used.
 - **DDS-Security** – secure data exchange and access control.

7.2.3 Application Layer

Where user applications use language-specific DDS APIs to build custom publishers and subscribers.

7.3 How DDS Works

Key concepts in DDS architecture:

- **Global Data Space (GDS):** The virtual shared memory/network where all participant data exchange occurs – data-centric and decoupled from application logic.
- **Domains:** Logical partitions within the GDS. Participants within a domain can discover and communicate with each other but are isolated from other domains.
- **Topics:** Named data channels within a domain, each with a name, data type, and QoS policies, bridging publishers and subscribers.
- **Publishers and Subscribers:** Domain participants that send (*publishers*) or receive (*subscribers*) data via topics.
- **Data Writers and Data Readers:** Components responsible for writing to / reading from a specific topic, respectively.
- **QoS:** Defines behavior of data exchange; can be set at topic, publisher, subscriber, reader, or writer level, individually or as QoS profiles.

7.4 Example Pub-Sub Workflow

Example: publishing and subscribing to a robot’s temperature reading.

1. **Define the Topic:** e.g., a **Temperature** topic with fields for value, timestamp, and sensor ID.
2. **Create a Publisher:** Set the **Domain ID** (e.g., 0), configure **QoS** policies for reliable delivery, set up a data writer for the **Temperature** topic, and begin publishing.
3. **Create a Subscriber:** Set the same Domain ID and QoS settings, define a data reader for the **Temperature** topic, and process incoming data.

8 List of DDS Vendors

Commercial DDS vendors typically offer enhanced performance, control, advanced security, and certification, and are preferred in mission-critical sectors (defense, aerospace, medicine). Open-source DDS is favored in research, education, and ROS 2-based robotics.

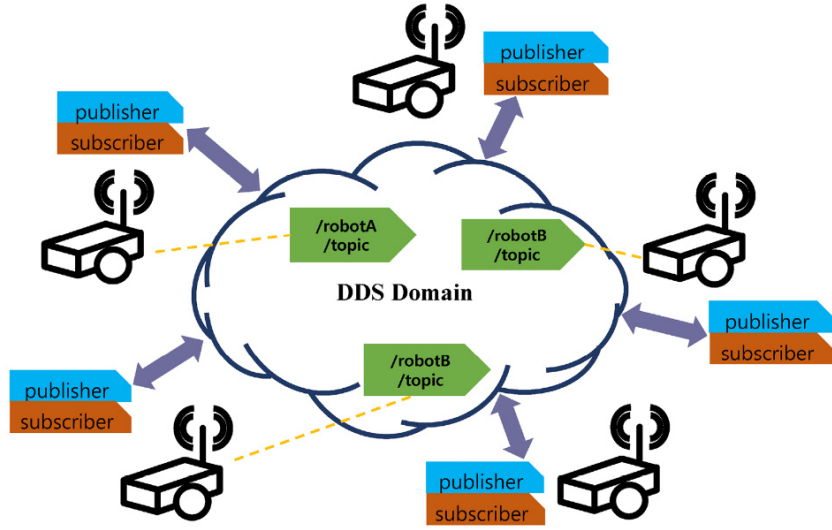


Figure 2: Multiple robots communicating via topics inside a DDS Domain.

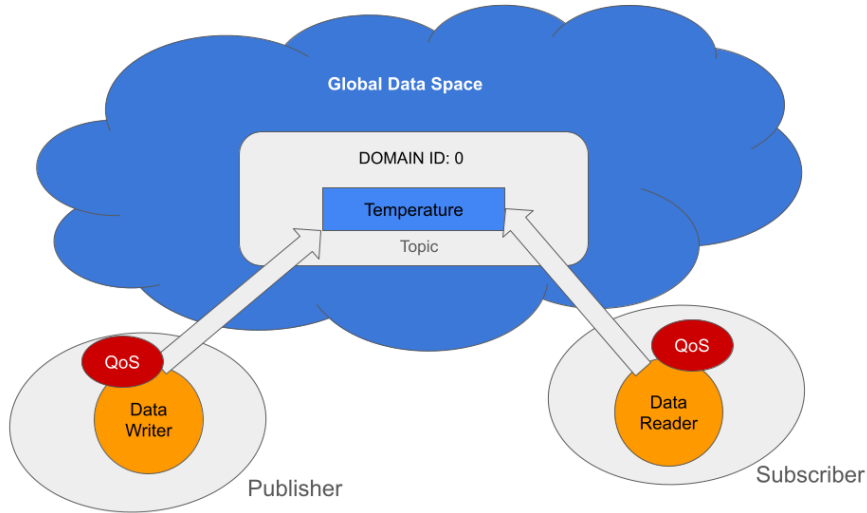


Figure 3: Example publish-subscribe workflow using DDS (Temperature topic, Domain ID 0).

Organization	DDS Name	Description / Core Features
Real-Time Innovations (RTI)	Connex DDS	Widely used, high performance and scalability, from embedded devices to the cloud; design/monitoring/analysis tools; strong documentation and interoperability.
ADLINK Technology	Vortex Splice	Open- High performance, reliability, extensive QoS and real-time data propagation; used in aerospace, defense, industrial automation.
Eclipse Foundation	Eclipse DDS	Cyclone Open-source, lightweight, efficient; designed for IoT and edge computing; multi-OS, community-driven.

Organization	DDS Name	Description / Core Features
Twin Oaks Computing	CoreDX DDS	Small footprint, high performance; suited for embedded/real-time and safety-critical systems; a lite version (CoreDX DDS Lite) is available.
eProsima	Fast DDS (formerly Fast RTPS)	Open-source, efficient and low latency; customizable QoS; integrates with other protocols via an integration service.
Object Computing, Inc. (OCI)	OpenDDS	Open-source, community-driven, customizable QoS, comprehensive DDS features, extensive documentation.
Zettascale	Cyclone DDS	Open-source, reliable and scalable for real-time distributed systems; small footprint, high throughput, low latency.

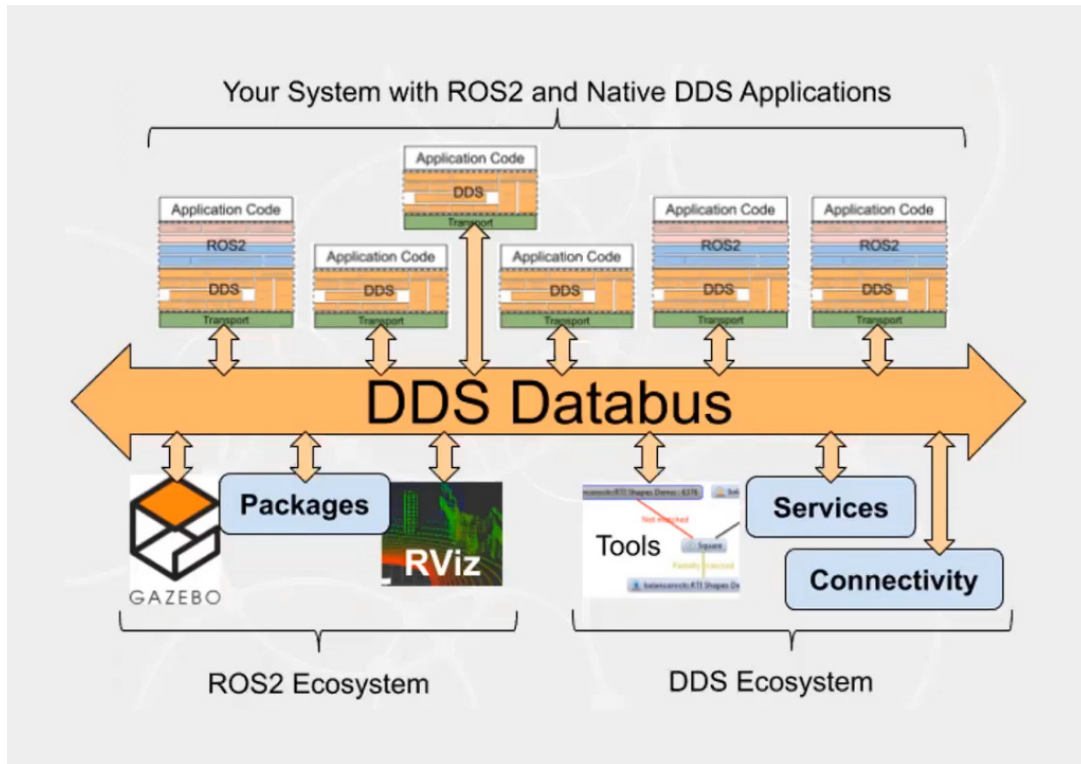


Figure 4: ROS 2 and native DDS applications sharing a common DDS Databus, connecting the ROS 2 ecosystem (packages, RViz, Gazebo) with the DDS ecosystem (tools, services, connectivity).

9 Layers Built on Top of DDS in ROS 2

9.1 RMW Layer (ROS Middleware Abstraction Interface)

The ROS 2 framework is built on top of the DDS API layer. The **RMW (ROS Middleware)** layer enables communication between ROS 2 nodes independent of the underlying middleware implementation, offering flexibility and interoperability by abstracting away DDS-specific details. There is a separate RMW implementation for each middleware vendor.

Popular RMW implementations:

- **rmw_fastrtps_cpp** – uses eProsima Fast DDS; high performance, low latency and high throughput.
- **rmw_cyclonedds_cpp** – uses Eclipse Cyclone DDS; suited for resource-constrained embedded/IoT devices.
- **rmw_connextdds_cpp** – uses RTI Connext; robust, feature-rich, used in mission-critical applications.
- **rmw_opensplice_cpp** – uses ADLINK’s OpenSplice DDS; high scalability and real-time capability.
- **rmw_zenoh** – uses **Zenoh**, a lightweight pub/sub protocol (not DDS) that performs comparably to DDS, particularly in bandwidth-constrained environments; selected by Open Robotics as an alternate ROS 2 middleware.
- **rmw_email** – a middleware that exchanges ROS 2 messages as strings over email.

9.2 RCL Layer and Client Libraries

RCL (ROS Client Library) connects the language-specific client libraries (**rclcpp** for C++, **rclpy** for Python, **rcl** for C) to the underlying RMW layer. Written in pure C, RCL provides stable APIs so that client libraries don’t need to interact directly with lower-level DDS APIs. Community-maintained client libraries also exist for Node.js, Rust, Dart, and C#.

9.3 ROS 2 Application Layer

The topmost layer, where ROS 2 **nodes** and **packages** reside, including navigation, manipulation, and perception capabilities.

In theory, ROS 2 developers can focus purely on the application layer without worrying about the DDS implementation underneath. In practice, however, deploying ROS 2 at large scale often still requires significant debugging effort – driving growing interest in alternatives such as **Zenoh**, selected as an alternative ROS 2 middleware.

10 Summary

This document introduced **ROS 2**, a widely used framework for robotics application development. Key takeaways:

- ROS provides modularity and reusability, letting developers build robotic applications quickly by reusing existing packages.
- Core features include IPC infrastructure, hardware abstraction, open-source licensing, and extensive tools/libraries for visualization, debugging, and communication.
- **ROS 2** improves on **ROS 1** through DDS-based middleware, enhanced security, real-time capability, and cross-platform compatibility (no single-point-of-failure master node).
- ROS evolved from Stanford AI Lab (2007) through Willow Garage and OSRF/Open Robotics to the current **ROS 2 Kilted Kaiju** (2025) release.
- **DDS** is the core communication middleware of ROS 2, built around Global Data Space, Domains, Topics, Publishers/Subscribers, Data Writers/Readers, and QoS policies – with vendors like RTI, ADLINK, eProsima, and Eclipse Cyclone DDS.
- The **RMW** and **RCL** layers abstract DDS details from application developers, enabling middleware interoperability (including newer alternatives like **Zenoh**).

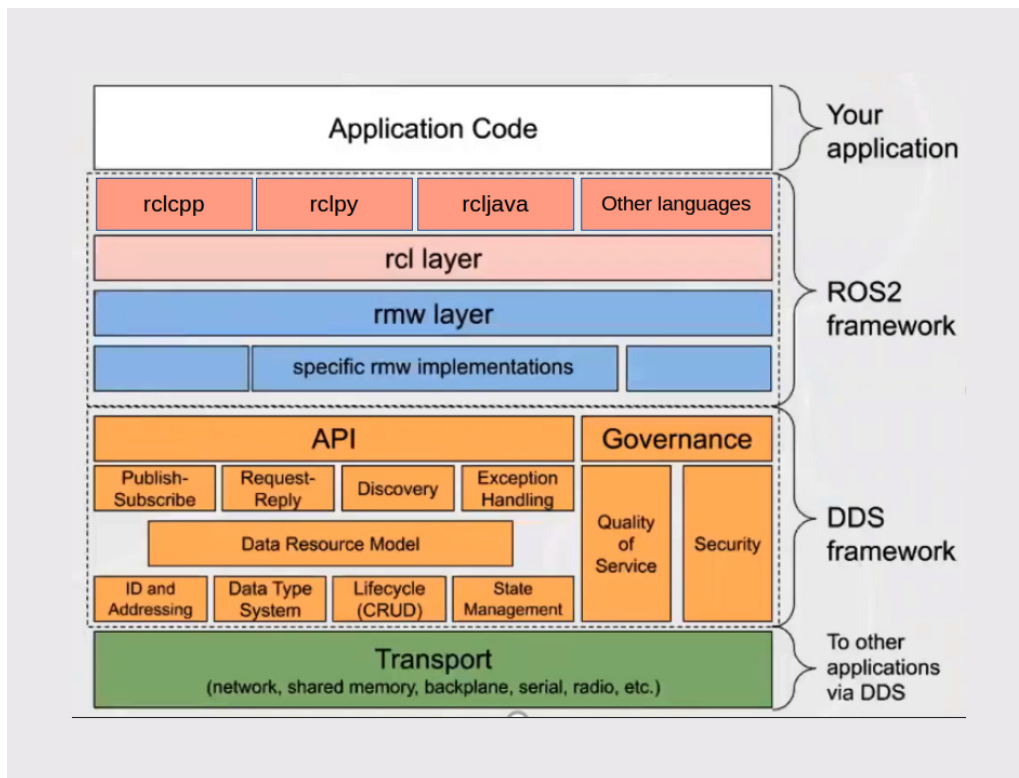


Figure 5: Full ROS 2 stack: application code, client libraries (rclcpp, rclpy, rcljava, etc.), the RCL and RMW layers, the underlying DDS framework (API, QoS, Security, Governance), and the transport layer.